OBJECT DETECTION · SET PREDICTION · REAL TIME

# NMS-Free Real-Time Detection

DETR → RT-DETR: moving duplicate suppression from post-processing into the loss, and the encoder bottleneck into the architecture.

**Central claim** — one-to-one Hungarian matching learns duplicate suppression, so NMS disappears. RT-DETR keeps that principle and re-engineers encoder, query learning, and decoder depth for real time.

**42.0**
DETR R50 AP · 41M params

**53.1**
RT-DETR R50 AP · 42M

**108**
FPS · T4 · TensorRT FP16

**34.8**
$AP_S$ · small-object gap

**Problem**
Anchors · duplicate boxes · NMS cost

› **DETR**
Set prediction · optimal one-to-one matching

› **RT-DETR**
Efficient encoder · better queries · tunable depth

Carion et al., ECCV 2020 — arXiv:2005.12872 · Lv et al., CVPR 2024 — arXiv:2304.08069
Code: github.com/facebookresearch/detr · github.com/lyuwenyu/rt-detr
Figures: our Day 07–08 measurements (synthetic, CPU) unless a paper table is cited.

## 01 · STARTING POINT

Conventional detectors emit many proposals or anchors and let NMS heuristics remove duplicates. DETR predicts the object set in one pass.

**Backbone features** › **Transformer global context** › **100 queries parallel decode**

### Matching before the loss

The Hungarian algorithm finds the globally optimal one-to-one assignment between prediction slots and ground-truth objects, so duplicates become structurally impossible.

$$L = L_{\text{class}} + \lambda_1 L_1 + \lambda_{\text{GIoU}} L_{\text{GIoU}}$$

Matching cost uses (bounded, commensurable with the box term); the loss uses . Under the same 15% relative error, L1 penalises a large box 9.44× more than a small one, while GIoU stays at 1.000×.

## 02 · THE NMS DILEMMA

Paper Table 1 (YOLOv8, T4 · TensorRT FP16): raising the confidence threshold removes candidates — faster but less accurate; raising the IoU threshold suppresses fewer boxes — more accurate but slower.

| CONTROL | AP | NMS |
|---|---|---|
| conf 0.001 → 0.05 | 52.9 → 51.2 | 2.36 → 1.06 ms |
| IoU 0.5 → 0.8 | 52.1 → 52.8 | 2.24 → 2.46 ms |

YOLO papers report model speed excluding NMS time, so RT-DETR builds an end-to-end benchmark (§3.2) with the NMS plugin included.

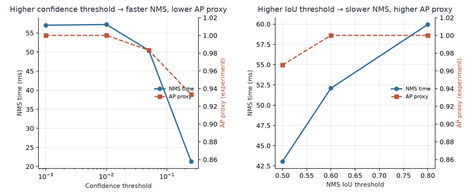



Fig. 1. NMS cost model — our measurement; direction matches paper Table 1.

## 03 · BOTTLENECK → DESIGN

In Deformable-DETR the encoder consumes 49% of GFLOPs for 11% of AP (a second-hand statistic, not our measurement). RT-DETR re-architects it:

**AIFI** self-attention on S5 only (400 tokens)

**CCFF** conv-based cross-scale fusion (linear cost)

Attention cost grows with $H^2$: 8400 candidate tokens vs 400 on S5 is a 441× gap. Variant D_S5: latency −35% and AP +0.4.

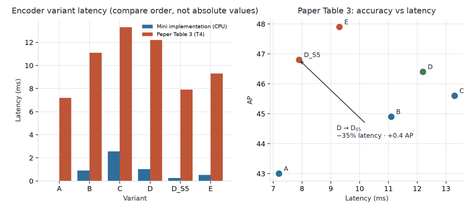



Fig. 2. Encoder variants — paper Table 3; our latency order reproduces it exactly.

## 04 · QUERY QUALITY

U = IP − CE — the mismatch between the location and classification prediction of a feature — enters the loss, so the encoder learns features that agree in both.

**AP** 47.9 → 48.7 **Prop cls** 0.35% → 0.82% **Prop both** 0.30% → 0.67%

Selection ≠ objective: the public top-K still ranks by class score (`rtdetr_decoder.py:491`) — uncertainty changes the loss, not the ranking.

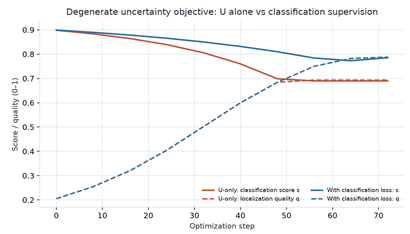



Fig. 3. Uncertainty alone has a degenerate optimum — our measurement.

## 05 · INFERENCE CONTROL

Auxiliary heads train every decoder layer, so inference can stop early without retraining. Paper Table 5 (Det 6): 6 layers 53.1 AP / 9.3 ms → 5 layers 53.0 / 8.8 ms → 1 layer 49.1 / 7.0 ms.

Cost per layer is nearly constant (0.4–0.5 ms) while the accuracy gain shrinks with depth (+2.2 AP at layer 1→2, only +0.1 at 5→6).

`eval_idx = which layer` `max_det = output cap` `lower max_det ≠ faster`

## 06 · RESULTS

| MODEL | AP | $AP_S$ | $AP_L$ | FPS |
|---|---|---|---|---|
| DETR R50 | 42.0 | 20.5 | | 28 |
| RT-DETR R50 | | 34.8 | | |

Same parameter count as Faster R-CNN-FPN+, but the scale profile flips: $AP_S$ −5.5, $AP_L$ +7.8. RT-DETR: +2.2 AP and 21× faster than DINO-Deformable (5 → 108 FPS).




Fig. 4. Accuracy vs real-time speed — paper Table 2 (NMS included for YOLO baselines).

## 07 · OPEN LIMITS

- Small objects stay weak — $AP_S$ 34.8 (–0.5 vs YOLOv8-L; R101 –0.9): the cost of restricting global attention to S5 while keeping multi-scale features.
- 49% / 11% is a second-hand statistic — cited, not measured by us.
- 136% / 120% are relative counts, not percentage-point gains.
- 2D COCO only — depth, BEV and multi-view remain untested.

## 08 · STUDY PROCESS

8 study days, 16 deep-dive documents, 4 runnable demos, 22 charts, 0 citation errors. Each claim was checked against the original table or equation, then probed with our own matching / NMS / attention / query-loss experiments.

Those checks are CPU, synthetic, direction-only — they are not reproductions of the paper's COCO/GPU numbers.

### Next

`GLIP + SAM` `CLIP` `LLaVA ×2` `VL-JEPA` `3D: DETR3D → PETR → Sparse4D`

Move duplicates into the loss · compute into the architecture · initialization into quality

## 09 · EVIDENCE AT A GLANCE

Four checks that support the design decisions, all reproduced locally:

**A** greedy vs optimal matching: duplicates 31.6%, greedy never wins

**B** attention cost: 8× tokens → 42.3× time (theory 64×)

**C** uncertainty alone: s 0.90 → 0.690 (degenerate)

**D** encoder latency order = paper order

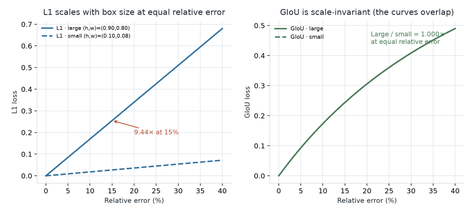



Fig. 5. L1 scales with box size, GIoU does not — our measurement.